

10.1.1 Subspace iteration with a Hermitian matrix

10.1.1 Subspace iteration, Part 1

fit width



The idea behind subspace iteration is to perform the Power Method with more than one vector in order to converge to (a subspace spanned by) the eigenvectors associated with a set of eigenvalues.

We continue our discussion by restricting ourselves to the case where $A \in \mathbb{C}^{m \times m}$ is Hermitian. Why? Because the eigenvectors associated with distinct eigenvalues of a Hermitian matrix are mutually orthogonal (and can be chosen to be orthonormal), which will simplify our discussion. Here we repeat the Power Method:

```
v0 := random vector
v0^(0) := v0 / ||v0||_2      normalize to have length one
for k := 0, ...
    v0 := A v0^(k)
    v0^(k+1) := v0 / ||v0||_2  normalize to have length one
endfor
```

In previous discussion, we used $v^{(k)}$ for the current approximation to the eigenvector. We now add the subscript to it, $v_0^{(k)}$, because we will shortly start iterating with multiple vectors.

Homework 10.1.1.1. You may want to start by executing `git pull` to update your directory `Assignments`.

Examine `Assignments/Week10/matlab/PowerMethod.m` which implements

```
[ lambda_0, v0 ] = PowerMethod( A, x, maxiters, illustrate, delay )
```

This routine implements the Power Method, starting with a vector `x` for a maximum number of iterations `maxiters` or until convergence, whichever comes first. To test it, execute the script in `Assignments/Week10/matlab/test_SubspaceIteration.m` which uses the Power Method to compute the largest eigenvalue (in magnitude) and corresponding eigenvector for an $m \times m$ Hermitian matrix A with eigenvalues $1, \dots, m$.

Be sure to click on "Figure 1" to see the graph that is created.

[Solution](#)

Watch the video regarding this problem on YouTube:

<https://youtu.be/8Bgf1tJeMng>. (embedding a video in a solution seems to cause PreTeXt trouble...)

10.1.1 Subspace iteration, Part 2

fit width



- Recall that when we analyzed the convergence of the Power Method, we commented on the fact that the method converges to an eigenvector associated with the largest eigenvalue (in magnitude) if two conditions are met:
- $|\lambda_0| > |\lambda_1|$.
 - $v_0^{(0)}$ has a component in the direction of the eigenvector, x_0 , associated with λ_0 .

A second initial vector, $v_1^{(0)}$, does not have a component in the direction of x_0 if it is orthogonal to x_0 . So, if we know x_0 , then we can pick a random vector, subtract out the component in the direction of x_0 , and make this our vector $v_1^{(0)}$ with which we should be able to execute the Power Method to find an eigenvector, x_1 , associated with the eigenvalue that has the second largest magnitude, λ_1 . If we then start the Power Method with this new vector (and don't introduce roundoff error in a way that introduces a component in the direction of x_0), then the iteration will home in on a vector associated with λ_1 (provided A is Hermitian, $|\lambda_1| > |\lambda_2|$, and $v_1^{(0)}$ has a component in the direction of x_1 .) This iteration would look like

```
x0 := x0 / ||x0||2           normalize known eigenvector x0 to have length one
v1 := random vector
v1 := v1 - x0^H v1 x0        make sure the vector is orthogonal to x0
v1^(0) := v1 / ||v1||2       normalize to have length one
for k := 0, ...
    v1 := A v1^(k)
    v1^(k+1) := v1 / ||v1||2   normalize to have length one
endfor
```

Homework 10.1.1.2. Copy `Assignments/Week10/matlab/PowerMethod.m` into `PowerMethodLambda1.m`. Modify it by adding an input parameter `v0`, which is an eigenvector associated with λ_0 (the eigenvalue with largest magnitude).

```
[ lambda_1, v1 ] = PowerMethodLambda1( A, x, x0, maxiters, illustrate, del
```

The new function should subtract this vector from the initial random vector as in the above algorithm.

Modify the appropriate line in `Assignments/Week10/matlab/test_SubspaceIteration.m`, changing (0) to (1), and use it to examine the convergence of the method.

What do you observe?

[▶ Solution](#)

10.1.1 Subspace iteration, Part 3

fit width

Because we should be concerned about the introduction of a component in the direction of x_0 due to roundoff error, we may want to reorthogonalize with respect to x_0 in each iteration:

```
x0 := x0 / ||x0||2           normalize known eigenvector x0 to have length one
v1 := random vector
v1 := v1 - x0^H v1 x0        make sure the vector is orthogonal to x0
v1^(0) := v1 / ||v1||2       normalize to have length one
for k := 0, ...
    v1 := A v1^(k)
    v1 := v1 - x0^H v1 x0     make sure the vector is orthogonal to x0
    v1^(k+1) := v1 / ||v1||2   normalize to have length one
endfor
```

Homework 10.1.1.3. Copy `PowerMethodLambda1.m` into `PowerMethodLambda1Reorth.m` and modify it to reorthogonalize with respect to `x0`:

```
[ lambda_1, v1 ] = PowerMethodLambda1Reorth( A, x, v0, maxiters, illustrat
```

Modify the appropriate line in `Assignments/Week10/matlab/test_SubspaceIteration.m`, changing `(0)` to `(1)`, and use it to examine the convergence of the method.

What do you observe?

[▶ Solution](#)

10.1.1 Subspace iteration, Part 4

fit width



We now observe that the steps that normalize x_0 to have unit length and then subtract out the component of v_1 in the direction of x_0 , normalizing the result, are exactly those performed by the Gram-Schmidt process. More generally, it is equivalent to computing the QR factorization of the matrix $\begin{pmatrix} x_0 & v_1 \end{pmatrix}$. This suggests the algorithm

```

v1 := random vector
(( x0 | v1^(0) ), R) := QR(( x0 | v1 ))
for k := 0, ...
    (( x0 | v1^(k+1) ), R) := QR(( x0 | Av1^(k) ))
endfor

```

Obviously, this redundantly normalizes x_0 . It puts us on the path of a practical algorithm for computing the eigenvectors associated with λ_0 and λ_1 .

The problem is that we typically don't know x_0 up front. Rather than first using the power method to compute it, we can instead iterate with two random vectors, where the first converges to a vector associated with λ_0 and the second to one associated with λ_1 :

```

v0 := random vector
v1 := random vector
(( v0^(0) | v1^(0) ), R) := QR(( v0 | v1 ))
for k := 0, ...
    (( v0^(k+1) | v1^(k+1) ), R) := QR(A ( v0^(k) | v1^(k) ))
endfor

```

We observe:

- If $|\lambda_0| > |\lambda_1|$, the vectors $v_0^{(k)}$ will converge linearly to a vector in the direction of x_0 at a rate dictated by the ratio $|\lambda_1|/|\lambda_0|$.
- If $|\lambda_0| > |\lambda_1| > |\lambda_2|$, the vectors $v_1^{(k)}$ will converge linearly to a vector in the direction of x_1 at a rate dictated by the ratio $|\lambda_2|/|\lambda_1|$.
- If $|\lambda_0| \geq |\lambda_1| > |\lambda_2|$ then $\text{Span}(\{v_0^{(k)}, v_1^{(k)}\})$ will eventually start approximating the subspace $\text{Span}(\{x_0, x_1\})$.

10.1.1 Subspace iteration, Part 5

fit width



What we have described is a special case of **subspace iteration**. The associated eigenvalue can be approximated via the Rayleigh quotient:

$$\lambda_0 \approx \lambda_0^{(k)} = v_0^{(k)H} A v_0^{(k)} \text{ and } \lambda_1 \approx \lambda_1^{(k)} = v_1^{(k)H} A v_1^{(k)}$$

Alternatively,

$$A^{(k)} = \begin{pmatrix} v_0^{(k)} & v_1^{(k)} \end{pmatrix}^H A \begin{pmatrix} v_0^{(k)} & v_1^{(k)} \end{pmatrix} \text{ converges to } \begin{pmatrix} \lambda_0 & 0 \\ 0 & \lambda_1 \end{pmatrix}$$

if A is Hermitian, $|\lambda_1| > |\lambda_2|$, and $v^{(0)}$ and $v^{(1)}$ have components in the directions of x_0 and x_1 , respectively.

The natural extension of these observations is to iterate with n vectors:

$$\begin{aligned} \hat{V} &:= \text{random } m \times n \text{ matrix} \\ (\hat{V}^{(0)}, R) &:= \text{QR}(\hat{V}) \\ A^{(0)} &= V^{(0)H} A V^{(0)} \\ \text{for } k &:= 0, \dots \\ &(\hat{V}^{(k+1)}, R) := \text{QR}(A \hat{V}^{(k)}) \\ A^{(k+1)} &= \hat{V}^{(k+1)H} A \hat{V}^{(k+1)} \\ \text{endfor} \end{aligned}$$

By extending the reasoning given so far in this unit, if

- A is Hermitian,
- $|\lambda_0| > |\lambda_1| > \dots > |\lambda_{n-1}| > |\lambda_n|$, and
- each v_j has a component in the direction of x_j , an eigenvector associated with λ_j ,

then each $v_j^{(j)}$ will converge to a vector in the direction x_j . The rate with which the component in the direction of x_p , $0 \leq p < n$, is removed from $v_j^{(k)}$, $n \leq j < m$, is dictated by the ratio $|\lambda_p|/|\lambda_j|$.

If some of the eigenvalues have equal magnitude, then the corresponding columns of $\hat{V}^{(k)}$ will eventually form a basis for the subspace spanned by the eigenvectors associated with those eigenvalues.

Homework 10.1.1.4. Copy `PowerMethodLambda1Reorth.m` into `SubspaceIteration.m` and modify it to work with an $m \times n$ matrix V :

```
[ Lambda, V ] = SubspaceIteration( A, V, maxiters, illustrate, delay );
```

Modify the appropriate line in

`Assignments/Week10/matlab/test_SubspaceIteration.m`, changing (0) to (1), and use it to examine the convergence of the method.

What do you observe?

▼ [Solution](#)

- `Assignments/Week10/answers/SubspaceIteration.m`

Watch the video regarding this problem on YouTube:

<https://youtu.be/Er7jGYs0HbE>. (embedding a video in a solution seems to cause PreTeXt trouble...)